

SMC CPU Cluster — CCI Integration Guide

- [SystemC CCI — What It Is, Why It Matters, and How to Use It in the SMC CPU Cluster](#)
 - [Table of Contents](#)
 - [1. What is CCI?](#)
 - [What CCI is NOT](#)
 - [2. Core Concepts](#)
 - [2.1 Parameter \(`cci\_param<T>` \)](#)
 - [2.2 Broker \(`cci\_broker\_if` / `cci\_broker\_handle` \)](#)
 - [2.3 Preset Value](#)
 - [2.4 Handle \(`cci\_param\_typed\_handle<T>` / `cci\_param\_untyped\_handle` \)](#)
 - [2.5 Originator \(`cci\_originator` \)](#)
 - [3. CCI Use Cases \(General\)](#)
 - [3.1 Use Case: Tool-Driven Pre-Simulation Configuration](#)
 - [3.2 Use Case: Regression / Sweep Testing](#)
 - [3.3 Use Case: Run-Time Introspection and Debugging](#)
 - [3.4 Use Case: Parameter Locking for Safety](#)
 - [3.5 Use Case: IP Packaging with Privacy](#)
 - [4. How CCI Applies to the CPU Cluster](#)
 - [4.1 Current State of Configuration in the CPU Cluster](#)
 - [4.2 Cluster Parameters Well-Suited for CCI](#)
 - [4.3 CPU-Cluster Testbench Scenarios Enabled by CCI](#)
 - [5. Concrete API Examples for the CPU Cluster](#)
 - [5.1 Declaring CCI Parameters Inside `smc\_cpu\_cluster`](#)
 - [5.2 Platform / Testbench Setup](#)
 - [5.3 Pre-Write Callback for Quantum Validation](#)
 - [5.4 Enumerate All Cluster Parameters \(Introspection\)](#)
 - [5.5 Hooking the AXI-Extension Source ID for Multi-Cluster Topologies](#)
 - [6. Benefits of Adopting CCI in This Project](#)
 - [6.1 Interoperability with EDA Tools](#)
 - [6.2 Separation of Configuration from Implementation](#)
 - [6.3 Testbench Flexibility Without Recompilation](#)
 - [6.4 Complete Parameter Provenance and Audit Trail](#)
 - [6.5 Validation and Safety via Callbacks](#)
 - [6.6 Documentation Embedded in the Model](#)
 - [6.7 Metadata for Advanced Tooling](#)
 - [6.8 Zero Overhead on Hot Paths](#)
 - [6.9 Standards Compliance Signals IP Maturity](#)
 - [7. Migration Path: `smc\_cpu\_cluster::config` → CCI Parameters](#)
 - [Phase 1 — Add CCI Parameters \(non-breaking\)](#)
 - [Phase 2 — Publish Parameters for Introspection](#)
 - [Phase 3 — Deprecate `config`](#)
 - [Phase 4 — Add Metadata and Callbacks](#)
 - [Phase 5 — Lift ELF Loading and Trace Path into CCI](#)
 - [8. Build Integration](#)

- [9. Summary](#)

SystemC CCI — What It Is, Why It Matters, and How to Use It in the SMC CPU Cluster

Document: 04_CCI_Integration_Guide.md **Project:** SMC CPU Cluster (SystemC/TLM-2.0 Loosely-Timed Model) **Standard Referenced:** SystemC CCI 1.0 LRM (Accellera, June 2018) **Companion docs:** - 01_SMC_Architecture.pdf §3 — IP role and configuration parameters - 02_SMC_IP_LowLevel_Design.pdf §3 — TLM interface, register map, AXI extension contract, `iss_hart` debug API - 03_CPU_Cluster_Test_Plan.md — verification strategy and the GoogleTest binaries that benefit from CCI-driven sweeps

Table of Contents

1. [What is CCI?](#)
 2. [Core Concepts](#)
 3. [CCI Use Cases \(General\)](#)
 4. [How CCI Applies to the CPU Cluster](#)
 5. [Concrete API Examples for the CPU Cluster](#)
 6. [Benefits of Adopting CCI in This Project](#)
 7. [Migration Path: `smc\_cpu\_cluster::config` → CCI Parameters](#)
 8. [Build Integration](#)
 9. [Summary](#)
-

1. What is CCI?

SystemC Configuration, Control and Inspection (CCI) is an Accellera-standardised C++ API layer (version 1.0, June 2018) built on top of IEEE Std 1666-2011 (SystemC). Its formal purpose is:

“To provide a standard for developing configurable SystemC models and supporting the development of configuration tools.” — CCI 1.0 LRM §1.2

In plain terms, CCI solves a pervasive problem in SystemC-based Virtual Platforms (VPs) and TLM models: **every IP block designer invented their own ad-hoc mechanism for exposing, setting, and introspecting model parameters**. The result was a proliferation of custom configuration structs, environment variables, command-line parsers, and constructor arguments — none of them interoperable.

CCI replaces this fragmentation with a **single, standardised, broker-mediated parameter infrastructure** that works the same way across models from different vendors and projects.

What CCI is NOT

- It is not a file format (no YAML/JSON schema is mandated; JSON serialisation is supported via `cci_value::to_json()` / `from_json()` but format choice is left to tools).
 - It is not a run-time configuration database replacement (it is a C++ library integrated at elaboration and simulation time).
 - It is not specific to any EDA tool — it is an open standard with a reference implementation.
-

2. Core Concepts

Understanding CCI requires familiarity with five key abstractions. The terminology is identical to the one used in `04_CCI_Integration_Guide.md` for the PLIC IP — only the example parameters change.

2.1 Parameter (`cci_param<T>`)

A **parameter** is a named, typed, configurable value owned by a `sc_module`. It is the fundamental unit of configuration.

```
cci_param<unsigned> num_harts_p{
    "num_harts",           // name (relative to enclosing module)
    1u,                   // default value
    "Number of RV64GC harts in the cluster (1..4)."
```

Parameters are:

- **Typed at compile time** (`cci_param<T>`) but also accessible via a type-erased variant (`cci_value`).
- **Registered automatically** with the nearest broker upon construction.
- Usable like normal variables (`p = p + 1; if (p > 2) { ... }`).
- Optionally **immutable** (`CCI_IMMUTABLE_PARAM`) or **locked** at run-time.

2.2 Broker (`cci_broker_if` / `cci_broker_handle`)

A **broker** is a registry that aggregates parameters and manages preset values. There is always one **global broker**; modules may optionally register **local brokers** to keep internal parameters private.

```
Global broker
├─ top.cluster.num_harts      ← exposed for tool override
├─ top.cluster.reset_pc
├─ top.cluster.quantum_ns
├─ top.plic.num_sources
└─ top.clint.num_harts
```

The broker is the point of contact for tools: a configuration file loader, a testbench stimulus generator, or a regression script all talk to the broker to inject preset values before modules elaborate.

2.3 Preset Value

A **preset value** is a value injected into the broker *before* the owning module is constructed. When the parameter's constructor runs, it automatically picks up the preset instead of its default.

```
broker.set_preset_cci_value("top.cluster.num_harts", cci_value(4u));
broker.set_preset_cci_value("top.cluster.reset_pc", cci_value(uint64_t(0xC0040000)));

smc::smc_cpu_cluster cluster("cluster");
// → num_harts_p == 4 and reset_pc_p == 0xC0040000, not the defaults
```

This is the primary mechanism for **tool-driven and testbench-driven configuration**.

2.4 Handle (cci_param_typed_handle<T> / cci_param_untyped_handle)

External code (testbenches, monitors, other modules) never touches a parameter directly. They obtain a **handle** from the broker via name lookup:

```
auto h = broker.get_param_handle<unsigned>("top.cluster.num_harts");
if (h.is_valid()) {
    unsigned n = h.get_value(); // typed read
}
```

Handles track originator identity, are automatically invalidated when the parameter is destroyed, and carry the same read/write API as the parameter itself.

2.5 Originator (cci_originator)

Every read and write through CCI carries an **originator** — the identity (sc_object pointer or string name) of whoever is making the access. This gives complete traceability:

“Parameter `cluster.quantum_insts` was changed from 1000 to 100 by `tb.sweep_driver`.”

3. CCI Use Cases (General)

The CCI LRM identifies five stakeholder perspectives. The table below maps them to concrete activities:

Perspective	What CCI Enables
Tools	EDA platforms enumerate all parameters in a simulation, render a configuration GUI, inject values from a YAML/JSON file, log changes.
IP Provider (model owner)	Declare parameters in the module; they are automatically visible to any compliant tool or testbench without custom glue code.
Testbench / verification	Programmatically set any parameter by name, register callbacks triggered on read/write, enumerate and dump all current values.
System integrator	Use local brokers to selectively expose/hide sub-module parameters; set per-instance preset values at the platform level.
Infrastructure	Adapt legacy parameter solutions (custom structs, environment variables) to the standard interface once, reuse everywhere.

3.1 Use Case: Tool-Driven Pre-Simulation Configuration

A configuration file is parsed and preset values are pushed into the global broker before `sc_start()`. Every module that uses CCI parameters picks them up automatically during elaboration — no module modification needed.

3.2 Use Case: Regression / Sweep Testing

A test harness iterates over a matrix of parameter values. For each configuration it:

1. Creates a fresh global broker.
2. Sets a batch of preset values.
3. Constructs the DUT.
4. Runs simulation.
5. Queries parameter values post-simulation for coverage logging.

This is exactly the workflow `04_CPU_Cluster_Test_Plan.md §11.3` envisions for the §A.4 coverage harness.

3.3 Use Case: Run-Time Introspection and Debugging

A monitor module registers a `post_write_callback` on any parameter. Whenever a testbench updates the cluster's `quantum_ns` or `amo_lock_detect` flag, the callback fires — enabling cycle-accurate logging without modifying the DUT model.

3.4 Use Case: Parameter Locking for Safety

A safety-sensitive parameter can be locked after elaboration so that no further modifications are accepted:

```
num_harts_p.lock(); // no password → permanently immutable for this run
```

3.5 Use Case: IP Packaging with Privacy

A sub-IP may expose only selected parameters to the integrator by registering a local broker:

```
cci_utils::broker localBroker("clusterBroker");
cci_register_broker(localBroker);
localBroker.expose.insert("cluster.num_harts"); // visible outside
// cluster.quantum_insts stays private to the cluster
```

4. How CCI Applies to the CPU Cluster

4.1 Current State of Configuration in the CPU Cluster

The cluster currently uses a plain C++ configuration struct passed at construction time (see

`libsmc/cpu/smc_cpu_cluster.h`):

```

class smc_cpu_cluster : public sc_core::sc_module {
public:
    struct config {
        unsigned    num_harts        = 1;           // 1..4
        uint64_t    hart_id_base     = 0;
        uint64_t    reset_pc         = 0x80000000;
        uint64_t    mem_size         = 1ull << 32;
        uint64_t    fast_mem_lo      = 0x00000000;
        uint64_t    fast_mem_hi      = 0x80000000;
        uint64_t    mmio_lo          = 0x80000000;
        uint64_t    mmio_hi          = 0x90000000;
        std::string isa              = "rv64imafdc";
        uint64_t    quantum_ns       = 1000;
        unsigned    quantum_insts    = 1000;
        bool        amo_lock_detect  = true;
        uint16_t    source_id        = 0x10;         // SMC_CPU_SOURCE_ID
        uint64_t    ctrl_size_bytes  = 0x2000;       // 8 KiB
        uint64_t    local_base_default = 0xC000'0000ull;
    };
    smc_cpu_cluster(sc_core::sc_module_name name, const config& cfg);
    // ...
};

```

The testbench instantiates it as:

```

auto cfg = smc_test::make_default_cluster_cfg(/*N=*/4);
smc::smc_cpu_cluster cluster("cluster", cfg);

```

Limitations of the current approach:

Limitation	Impact
Configuration is opaque to tools	No EDA tool can enumerate or override cluster parameters without reading C++ source.
No run-time introspection	A testbench cannot query <code>num_harts</code> , <code>quantum_ns</code> , or <code>mmio_lo</code> by name at run-time.
No change traceability	Nothing records who set <code>quantum_insts = 16</code> or <code>amo_lock_detect = false</code> .
No preset mechanism	To change defaults, the caller must construct a custom <code>config</code> — no late-binding from a file.
No callback support	A monitor cannot react to changes in <code>source_id</code> or <code>local_base_default</code> without modifying the DUT.
<code>quantum_ns</code> / <code>quantum_insts</code> are baked in at construction	Sweep regressions require rebuilding the binary unless someone hand-rolls a CLI parser.
No parameter descriptions	Tool introspection has no human-readable documentation for any cluster knob.

4.2 Cluster Parameters Well-Suited for CCI

Every field of `smc_cpu_cluster::config` is a natural CCI parameter. The table below picks out the ones with the highest tooling / verification leverage:

CCI Parameter Name	Type	Default	Description
<code>cluster.num_harts</code>	<code>unsigned</code>	<code>1</code>	Number of RV64GC harts (1..4). Drives <code>sc_vector</code> widths.
<code>cluster.hart_id_base</code>	<code>uint64_t</code>	<code>0</code>	First <code>MHARTID</code> value; subsequent harts get <code>+1</code> each.
<code>cluster.reset_pc</code>	<code>uint64_t</code>	<code>0x80000000</code>	Per-hart reset vector (also writable via <code>RESET_VECTOR_N</code>).
<code>cluster.mem_size</code>	<code>uint64_t</code>	<code>0x1_0000_0000</code>	Sparse memory footprint allocated by Whisper's Memory.
<code>cluster.fast_mem_lo</code>	<code>uint64_t</code>	<code>0x00000000</code>	Lower bound of the wrapper's fast-mem window.
<code>cluster.fast_mem_hi</code>	<code>uint64_t</code>	<code>0x80000000</code>	Upper bound of the fast-mem window.
<code>cluster.mmio_lo</code>	<code>uint64_t</code>	<code>0x80000000</code>	Lower bound of <code>mmio</code> socket carve-out (§3.6).
<code>cluster.mmio_hi</code>	<code>uint64_t</code>	<code>0x90000000</code>	Upper bound of <code>mmio</code> socket carve-out.
<code>cluster.isa</code>	<code>string</code>	<code>"rv64imafdc"</code>	Whisper <code>configIsa</code> string.
<code>cluster.quantum_ns</code>	<code>uint64_t</code>	<code>1000</code>	TLM LT global quantum in nanoseconds.
<code>cluster.quantum_insts</code>	<code>unsigned</code>	<code>1000</code>	K in <code>step(K)</code> per scheduling slice (§3.5).
<code>cluster.amo_lock_detect</code>	<code>bool</code>	<code>true</code>	When true, <code>tlm_access</code> sets <code>prot[3] = 1</code> for AMO / LR-SC.
<code>cluster.source_id</code>	<code>uint16_t</code>	<code>0x10</code>	<code>smc_axi_extension.source_id</code> (= <code>SMC_CPU_SOURCE_ID</code>).
<code>cluster.ctrl_size_bytes</code>	<code>uint64_t</code>	<code>0x2000</code>	Size of the CPU-Control register window (§3.8).
<code>cluster.local_base_default</code>	<code>uint64_t</code>	<code>0xC000_0000</code>	Reset value of the RO <code>LOCAL_BASE</code> register.
<code>cluster.elf_paths</code>	<code>string[]</code>	<code>[]</code>	ELF files to load before <code>sc_start()</code> ; resolved via <code>loadElfFiles</code> .
<code>cluster.trace_path</code>	<code>string</code>	<code>" "</code>	Optional Whisper commit-log path (lockstep co-sim).

`elf_paths` and `trace_path` are not part of today's `config` struct — they are natural additions once CCI is in place, replacing the imperative `cluster.load_elf({...})` call with a declarative broker-driven setup.

4.3 CPU-Cluster Testbench Scenarios Enabled by CCI

Once parameters are CCI-managed, the testbench gains powerful new capabilities:

a) Name-based parameter override without recompilation:


```
// Loaded from a JSON file or CLI before the DUT is constructed
broker.set_preset_cci_value("cluster.num_harts",      cci_value(4u));
broker.set_preset_cci_value("cluster.quantum_insts",  cci_value(16u));
broker.set_preset_cci_value("cluster.amo_lock_detect", cci_value(false));
smc::smc_cpu_cluster cluster("cluster");
// → 4 harts, K=16 step, AMO-lock detection off
```

b) Post-simulation parameter dump for traceability:

```
for (auto& h : broker.get_param_handles()) {
    if (std::string(h.name()).find("cluster.") != 0) continue;
    std::cout << h.name() << " = " << h.get_cci_value()
               << " (origin: " << h.get_value_origin().name() << ")\n";
}
```

c) Callback-driven verification of the quantum:

```
auto h = broker.get_param_handle<uint64_t>("cluster.quantum_ns");
h.register_post_write_callback([](const cci_param_write_event<uint64_t>& ev) {
    SC_REPORT_INFO("cluster_tb",
        ("quantum_ns changed: " + std::to_string(ev.new_value)).c_str());
});
```

d) Regression sweep over hart counts and quantum settings:

```
for (unsigned n : {1u, 2u, 4u}) {
    for (uint64_t qns : {100u, 1000u, 10000u}) {
        cci::cci_register_broker(new cci_utils::consuming_broker("Global"));
        auto br = cci::cci_get_global_broker(cci::cci_originator("sweep"));
        br.set_preset_cci_value("cluster.num_harts", cci_value(n));
        br.set_preset_cci_value("cluster.quantum_ns", cci_value(qns));
        run_cluster_test();
    }
}
```

This is **exactly** the cross-product workflow that 04_CPU_Cluster_Test_Plan.md §9 calls for.

e) Declarative ELF loading instead of imperative `load_elf()`:

```
broker.set_preset_cci_value("cluster.elf_paths",
    cci_value(std::vector<std::string>{ "fw/bootrom.elf", "fw/payload.elf" }));
smc::smc_cpu_cluster cluster("cluster");
// → ELF's loaded automatically inside the constructor;
// tohost/fromhost symbol resolution happens before sc_start()
```

5. Concrete API Examples for the CPU Cluster

5.1 Declaring CCI Parameters Inside `smc_cpu_cluster`

```
// smc_cpu_cluster.h (modified)
#include <cci_configuration>

namespace smc {

class smc_cpu_cluster : public sc_core::sc_module {
public:
    // ... ports unchanged ...

    SC_HAS_PROCESS(smc_cpu_cluster);
    explicit smc_cpu_cluster(sc_core::sc_module_name name)
        : sc_core::sc_module(name)
        , num_harts_p("num_harts", 1u,
            "Number of RV64GC harts (1..4). Drives sc_vector widths.")
        , reset_pc_p("reset_pc", uint64_t(0x80000000),
            "Per-hart reset vector. Override per hart via RESET_VECTOR_N[i].")
        , isa_p("isa", std::string("rv64imafdc"),
            "Whisper configIsa string.")
        , quantum_ns_p("quantum_ns", uint64_t(1000),
            "TLM LT global quantum in nanoseconds.")
        , quantum_insts_p("quantum_insts", 1000u,
            "K in step(K) per scheduling slice (§3.5).")
        , amo_lock_detect_p("amo_lock_detect", true,
            "When true, tlm_access sets prot[3]=1 for AMO / LR-SC.")
        , source_id_p("source_id", uint16_t(0x10),
            "smc_axi_extension.source_id (= SMC_CPU_SOURCE_ID).")
        , fast_mem_lo_p("fast_mem_lo", uint64_t(0x0),
            "Lower bound of the wrapper's fast-mem window.")
        , fast_mem_hi_p("fast_mem_hi", uint64_t(0x80000000),
            "Upper bound of the fast-mem window.")
        , mmio_lo_p("mmio_lo", uint64_t(0x80000000),
            "Lower bound of the mmio socket carve-out (§3.6).")
        , mmio_hi_p("mmio_hi", uint64_t(0x90000000),
            "Upper bound of the mmio socket carve-out.")
        , ctrl_size_bytes_p("ctrl_size_bytes", uint64_t(0x2000),
            "Size of the CPU-Control register window (§3.8).")
        , local_base_default_p("local_base_default", uint64_t(0xC000'0000),
            "Reset value of the R0 LOCAL_BASE register.")
    {
        SC_METHOD(validate_cfg);
        sensitive << sc_core::SC_ZERO_TIME;
    }

private:
    cci::cci_param<unsigned>    num_harts_p;
    cci::cci_param<uint64_t>    reset_pc_p;
    cci::cci_param<std::string> isa_p;
    cci::cci_param<uint64_t>    quantum_ns_p;
    cci::cci_param<unsigned>    quantum_insts_p;
    cci::cci_param<bool>        amo_lock_detect_p;
    cci::cci_param<uint16_t>    source_id_p;
};
}
```

```
cci::cci_param<uint64_t>    fast_mem_lo_p;
cci::cci_param<uint64_t>    fast_mem_hi_p;
cci::cci_param<uint64_t>    mmio_lo_p;
cci::cci_param<uint64_t>    mmio_hi_p;
cci::cci_param<uint64_t>    ctrl_size_bytes_p;
cci::cci_param<uint64_t>    local_base_default_p;

void validate_cfg() {
    SC_REPORT_INFO("smc_cpu_cluster",
        ("num_harts=" + std::to_string(static_cast<unsigned>(num_harts_p)) +
        " quantum_ns=" + std::to_string(static_cast<uint64_t>(quantum_ns_p)) +
        " quantum_insts=" + std::to_string(static_cast<unsigned>
        (quantum_insts_p))).c_str());
    if (num_harts_p < 1 || num_harts_p > 4)
        SC_REPORT_FATAL("smc_cpu_cluster", "num_harts out of range [1..4]");
    if (fast_mem_hi_p < fast_mem_lo_p)
        SC_REPORT_FATAL("smc_cpu_cluster", "fast_mem window is inverted");
    if (mmio_hi_p < mmio_lo_p)
        SC_REPORT_FATAL("smc_cpu_cluster", "mmio window is inverted");
    if (quantum_insts_p == 0)
        SC_REPORT_FATAL("smc_cpu_cluster", "quantum_insts must be > 0");
}
};

} // namespace smc
```

5.2 Platform / Testbench Setup

```
// cluster_sweep_test.cpp (new)
#include <cci_configuration>
#include <cci/utils/consuming_broker.h>

int sc_main(int argc, char* argv[]) {
    // 1. Register global broker (required before any cci_param is constructed)
    cci::cci_register_broker(new cci_utils::consuming_broker("GlobalBroker"));

    // 2. Inject preset values (optional – override defaults before DUT exists)
    cci::cci_originator cfg_tool("config_loader");
    auto broker = cci::cci_get_global_broker(cfg_tool);
    broker.set_preset_cci_value("cluster.num_harts", cci::cci_value(4u));
    broker.set_preset_cci_value("cluster.reset_pc",
        cci::cci_value(uint64_t(0xC0040000)));
    broker.set_preset_cci_value("cluster.quantum_insts", cci::cci_value(16u));
    broker.set_preset_cci_value("cluster.amo_lock_detect", cci::cci_value(false));

    // 3. Instantiate DUT – picks up presets automatically
    smc::smc_cpu_cluster cluster("cluster");

    // 4. Verify via handle
    auto h_n = broker.get_param_handle<unsigned>("cluster.num_harts");
    assert(h_n.is_valid() && h_n.get_value() == 4u);

    // 5. Run the regression
    sc_core::sc_start();
    return 0;
}
```

5.3 Pre-Write Callback for Quantum Validation

```
// Reject pathological quantum_ns values BEFORE they take effect.
auto h = cci::cci_get_broker().get_param_handle<uint64_t>("cluster.quantum_ns");
h.register_pre_write_callback(
    [] (const cci::cci_param_write_event<uint64_t>& ev) {
        if (ev.new_value == 0 || ev.new_value > 1'000'000'000ull) {
            SC_REPORT_WARNING("cluster_tb",
                "cluster.quantum_ns out of sane range [1..1e9] ns");
        }
    }
);
```

5.4 Enumerate All Cluster Parameters (Introspection)

```
auto broker = cci::cci_get_broker();
for (auto& h : broker.get_param_handles()) {
    std::string name = h.name();
    if (name.find("cluster.") != 0) continue;
    std::cout << std::left << std::setw(40) << name
              << " = " << h.get_cci_value()
              << " [" << h.get_description() << "]\n";
}
```

5.5 Hooking the AXI-Extension Source ID for Multi-Cluster Topologies

```
// In a hypothetical dual-cluster SiP, each cluster gets its own source_id.
broker.set_preset_cci_value("cluster0.source_id", cci::cci_value(uint16_t(0x10)));
broker.set_preset_cci_value("cluster1.source_id", cci::cci_value(uint16_t(0x11)));
```

Downstream `axi_filter` instances can now distinguish the two clusters without any change to `smc_cpu_cluster` or to the filter code.

6. Benefits of Adopting CCI in This Project

6.1 Interoperability with EDA Tools

Any compliant SystemC simulator or platform tool (e.g., Synopsys Virtualizer, Cadence Xcelium VP) can enumerate, display, and override cluster parameters from a configuration GUI or scripting interface — without touching cluster source code. The model becomes a true plug-and-play IP component, on par with the PLIC and other SMC IPs.

6.2 Separation of Configuration from Implementation

Today, changing `num_harts` from 1 to 4 (or sweeping `quantum_insts` from 1 to 1000) requires modifying `smc_cpu_cluster::config` and recompiling the test binary. With CCI, the value is injected at platform-level or via a config file, and the cluster itself stays unchanged. This separates **model implementation** from **platform-specific configuration**, reducing maintenance burden.

6.3 Testbench Flexibility Without Recompile

CCI enables parameter sweeps, corner-case tests, and regressions driven entirely from test scripts or JSON configuration files. Different test scenarios can exercise different cluster topologies without recompiling the model:

```
# Hypothetical config file (CCI tool parses and calls set_preset_cci_value)
cluster.num_harts      = 4
cluster.reset_pc       = 0xC0040000
cluster.quantum_insts  = 16
cluster.amo_lock_detect = false
cluster.elf_paths      = [ "fw/bootrom.elf", "fw/payload.elf" ]
```

This is exactly the workflow the §A.4 / §B.3 sweeps in the SMC test plan envision.

6.4 Complete Parameter Provenance and Audit Trail

Every parameter write records *who* changed it and *from what value*. This is invaluable during debug:

- “Was `quantum_ns` set to 1 μ s by the testbench or by a CLI override?”
- “Which configuration loaded `amo_lock_detect = false`?”
- “Which file last touched `cluster.source_id`?”

`h.get_value_origin().name()` answers these questions directly.

6.5 Validation and Safety via Callbacks

Pre-write callbacks can enforce invariants centrally — e.g., reject `num_harts > 4`, inverted fast-mem / mmio windows, or `quantum_insts == 0` before the value is committed — rather than scattering validation across `smc_cpu_cluster.cpp` and `tlm_access()`.

6.6 Documentation Embedded in the Model

Parameter descriptions are first-class citizens of the CCI API. A tool calling `h.get_description()` retrieves “K in step(K) per scheduling slice (§3.5).” for `quantum_insts` — removing the need to chase down comments in `smc_cpu_cluster.h`.

6.7 Metadata for Advanced Tooling

CCI parameters support attaching arbitrary key-value metadata:

```
num_harts_p.add_metadata("ll_design_ref", cci::cci_value("02_SMC_IP_LowLevel_Design.pdf §3.4"));
num_harts_p.add_metadata("rdl_field", cci::cci_value("CORE_ENABLE[3:0]"));
num_harts_p.add_metadata("range_min", cci::cci_value(1u));
num_harts_p.add_metadata("range_max", cci::cci_value(4u));

source_id_p.add_metadata("fw_define", cci::cci_value("SMC_CPU_SOURCE_ID"));
source_id_p.add_metadata("axi_ext_field", cci::cci_value("smc_axi_extension.source_id"));
```

This metadata is queryable at run-time, enabling documentation generators, hardware abstraction layers, and coverage tools to consume it automatically.

6.8 Zero Overhead on Hot Paths

CCI parameter access uses C++ typed references under the hood. The `operator=` and arithmetic operators on `cci_param<T>` resolve to the underlying `T` directly. For simulation-critical paths (e.g., the per-hart `step(K)` loop or the memory-callback fast-mem branch), reading a CCI parameter into a local variable once per quantum is as fast as a plain variable read after the first elaboration.

6.9 Standards Compliance Signals IP Maturity

Adopting CCI signals to downstream integrators and EDA partners that the CPU Cluster model follows the same industry standard as the PLIC and (eventually) the rest of the SMC IP library. This reduces integration

friction and audit risk when the model is shared across teams or shipped to customers.

7. Migration Path: `smc_cpu_cluster::config` → CCI Parameters

The migration is incremental and non-breaking. The existing `config` struct can remain for backward compatibility while CCI parameters are introduced alongside it.

Phase 1 — Add CCI Parameters (non-breaking)

Keep the `config` constructor. Add `cci_param` members initialised from the struct:

```
smc_cpu_cluster(sc_core::sc_module_name name, const config& cfg = config{})
    : sc_core::sc_module(name)
    , num_harts_p("num_harts", cfg.num_harts, "...")
    , reset_pc_p("reset_pc", cfg.reset_pc, "...")
    , quantum_ns_p("quantum_ns", cfg.quantum_ns, "...")
    , quantum_insts_p("quantum_insts", cfg.quantum_insts, "...")
    , amo_lock_detect_p("amo_lock_detect", cfg.amo_lock_detect, "...")
    // ...
    {}
```

Existing GoogleTest binaries continue to work unchanged. New tests can use the CCI preset mechanism.

Phase 2 — Publish Parameters for Introspection

Register a global broker in each test binary's `sc_main`. No cluster source changes are needed.

Phase 3 — Deprecate `config`

Once all callers use CCI presets, remove `config` from the public API. The cluster constructor becomes:

```
explicit smc_cpu_cluster(sc_core::sc_module_name name);
```

All configuration flows through the broker. `test/include/smc_test_utils.h` gains a `make_default_cluster_presets()` helper that registers the canonical 4-hart RV64GC preset bundle.

Phase 4 — Add Metadata and Callbacks

Enrich parameters with `add_metadata` calls (linking back to `02_SMC_IP_LowLevel_Design.pdf` section anchors) and validation callbacks (reject inverted memory windows, zero `quantum_insts`, etc.) This is purely additive.

Phase 5 — Lift ELF Loading and Trace Path into CCI

Replace the imperative `cluster.load_elf({...})` and trace-file plumbing with declarative `cluster.elf_paths` / `cluster.trace_path` parameters resolved by the constructor before `sc_start()`.

8. Build Integration

CCI integration into the existing CMake build mirrors the PLIC recipe in `04_CCI_Integration_Guide.md §8`.

```
# In cpu_cluster/CMakeLists.txt (top-level)

# Point CMake to the CCI installation (override via -DCCI_HOME=...).
set(CCI_HOME "$ENV{CCI_HOME}" CACHE PATH "Path to SystemC CCI installation")
if(NOT CCI_HOME OR NOT EXISTS "${CCI_HOME}/include/cci_configuration")
    message(FATAL_ERROR
        "CCI header not found at CCI_HOME=${CCI_HOME}. "
        "Set -DCCI_HOME=<path> or export CCI_HOME.")
endif()

# Imported target for downstream linking.
add_library(SystemC::cci INTERFACE IMPORTED)
set_target_properties(SystemC::cci PROPERTIES
    INTERFACE_INCLUDE_DIRECTORIES "${CCI_HOME}/include"
    INTERFACE_LINK_LIBRARIES
        "${CCI_HOME}/lib/libcci-config.so;${CCI_HOME}/lib/libcci-inspection.so")

# Link the cluster library against CCI.
target_link_libraries(smc_cpu_cluster PUBLIC SystemC::cci)
```

Update `run_tests.sh` to probe / honour `CCI_HOME` the same way it already probes `SYSTEMC_HOME` and `WHISPER_HOME`, and prepend `${CCI_HOME}/lib` to `LD_LIBRARY_PATH` so the dynamic CCI libraries are visible at run time.

9. Summary

Topic	Current CPU Cluster	With CCI
Configuration mechanism	<code>smc_cpu_cluster::config</code> struct passed at construction	<code>cci_param<T></code> members auto-registered with broker
Tool visibility	None — opaque C++ struct	Full: enumerate, read, write, describe via broker API
Preset injection	Caller must build custom <code>config</code> instance	<code>broker.set_preset_cci_value()</code> before DUT construction
Run-time introspection	Not possible	<code>broker.get_param_handles()</code> → full name/value/description
Change audit trail	None	<code>h.get_value_origin().name()</code> → who set it
Validation	Hand-rolled checks in <code>smc_cpu_cluster.cpp</code>	Pre-write callbacks; centralised, reusable validators
Documentation	Header comments only	First-class descriptions in the parameter object
Metadata	None	Arbitrary key-value map attached to each parameter (LL-design refs, RDL fields, etc.)
ELF loading	Imperative <code>cluster.load_elf({...})</code>	Declarative <code>cluster.elf_paths</code> preset, resolved in ctor
Sweep regressions	Rebuild + relink per scenario	Preset-only; identical binary across the sweep
Build dependency	None	Link against <code>SystemC::cci</code> (CCI 1.0 install)
Backward compatibility	—	Phase 1 migration keeps <code>config</code> intact

CCI turns the CPU Cluster model from a self-contained C++ object with hard-coded configuration into a **standard-compliant, introspectable, tool-aware IP block** that can be configured, monitored, and validated consistently across any platform that supports SystemC — exactly the same posture the PLIC model is aiming for under `04_CCI_Integration_Guide.md`.